

Database Design Document

Site Doctor

Version 1.0

July 23, 2026

Contents

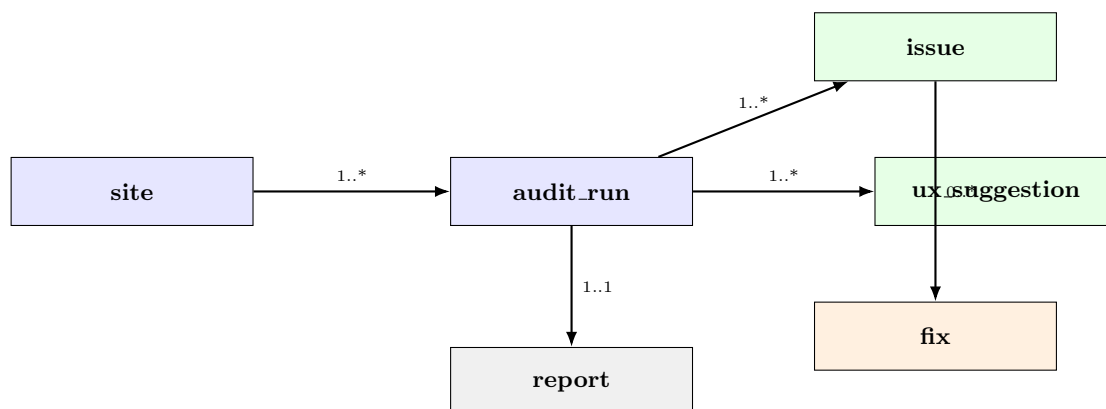
1	Purpose and Scope	3
2	Entity-Relationship Diagram	3
3	Entity Descriptions	3
4	Table Definitions	4
4.1	site	4
4.2	audit_run	4
4.3	issue	4
4.4	ux_suggestion	4
4.5	fix	5
4.6	report	5
5	Relationships	5
6	Indexes	6
7	Normalization	6
8	Constraints	6

1 Purpose and Scope

This document specifies the relational persistence layer for Site Doctor. The core audit pipeline itself is stateless per run — all cross-node data lives in an in-memory LangGraph state object during execution (see the Pydantic schemas in `models/schemas.py`). The schema described here is the storage layer that sits *underneath* that pipeline: it persists completed run reports so they can be listed, retrieved, and downloaded later (SRS requirement FR-20), independent of whether the pipeline process that produced them is still running.

A deliberate design decision carried through this schema: mechanically verifiable findings (`issue`) and judgment-based findings (`ux_suggestion`) are stored in separate tables rather than one polymorphic "finding" table, mirroring the same distinction enforced in the pipeline's own data model. This keeps every column's constraints meaningful — a mechanically-verified table with a nullable "observation" column (only meaningful for UX findings), or vice versa, would blur a distinction the rest of the system depends on.

2 Entity-Relationship Diagram



3 Entity Descriptions

Entity	Description
site	A distinct target URL that has been audited at least once. Exists independently of any single run so run history can be grouped per site.
audit_run	One execution of the pipeline against a site, with a fixed <code>selected_checks</code> configuration and lifecycle status.
issue	A single mechanically verifiable finding from the SEO/rule-based audit branch, scoped to one run.
ux_suggestion	A single judgment-based finding from the vision-review branch, scoped to one run.
fix	A proposed, and possibly applied and verified, remediation for a specific <code>issue</code> . Never associated with a <code>ux_suggestion</code> .
report	The generated, downloadable summary artifact for one completed run.

4 Table Definitions

4.1 site

Column	Type	Notes
id	UUID (PK)	Primary key, generated server-side.
url	TEXT	Normalized target URL (scheme + host + path, query string stripped).
created_at	TIMESTAMPTZ	First time this URL was audited.

4.2 audit_run

Column	Type	Notes
id	UUID (PK)	
site_id	UUID (FK → site.id)	
selected_checks	TEXT[]	Subset of {seo, ux, security}; security only present when the user explicitly opted in (SRS FR-04).
started_at	TIMESTAMPTZ	
completed_at	TIMESTAMPTZ, nullable	Null while the run is in progress.
status	TEXT	running / completed / failed.

4.3 issue

Column	Type	Notes
id	UUID (PK)	
run_id	UUID (FK → audit_run.id)	
lighthouse_audit_id	TEXT	Stable rule identifier (e.g. <code>image-alt</code>) from the tracked-audits map.
category	TEXT	seo / accessibility / performance.
severity	TEXT	high / medium / low, set during triage.
plain_language_summary	TEXT	

4.4 ux_suggestion

Column	Type	Notes
id	UUID (PK)	
run_id	UUID (FK → audit_run.id)	
category	TEXT	clutter / cta-overload / hierarchy / trust-signal, etc.
severity	TEXT	high / medium / low.
observation	TEXT	What the vision model observed.

Column	Type	Notes
recommendation	TEXT	What the model recommends changing.

4.5 fix

Column	Type	Notes
id	UUID (PK)	
issue_id	UUID (FK → issue.id), NOT NULL	Enforces that fixes only ever attach to mechanically verifiable issues.
before_snippet	TEXT	
after_snippet	TEXT	
approved	BOOLEAN, nullable	Null until the human decides.
applied	BOOLEAN	
verified_cleared	BOOLEAN, nullable	Null until reaudit runs.
attempts	INTEGER	Bounded by the pipeline's <code>max_retries_per_fix</code> setting.

4.6 report

Column	Type	Notes
id	UUID (PK)	
run_id	UUID (FK → audit_run.id), UNIQUE	One report per run.
scores_json	JSONB	Category scores at time of report generation.
generated_at	TIMESTAMPTZ	

5 Relationships

Relationship	Cardinality	Notes
site → audit_run	1 : N	A site accumulates one run per audit invocation, enabling history over time.
audit_run → issue	1 : N	Zero issues if the SEO check was not selected for that run.
audit_run → ux_suggestion	1 : N	Zero suggestions if the UX check was not selected for that run.
issue → fix	1 : N	Zero or more proposed fixes per issue (more than one only if earlier attempts failed verification and were retried).
audit_run → report	1 : 1	Exactly one report per completed run.

6 Indexes

Index	Purpose
UNIQUE on <code>site(url)</code>	Prevents duplicate site rows for the same normalized URL; enables upsert-on-crawl.
<code>audit_run(site_id)</code>	Supports "history for this site" queries.
<code>issue(run_id)</code>	Supports the primary "assemble a report" fan-out query.
<code>ux_suggestion(run_id)</code>	Same purpose, for the UX side.
<code>fix(issue_id)</code>	Supports fetching all fix attempts for a given issue.
UNIQUE on <code>report(run_id)</code>	Enforces the 1:1 run-to-report relationship at the schema level, not just application logic.

7 Normalization

The schema is designed to Third Normal Form (3NF):

- Every non-key column depends only on its own table's primary key (e.g. `issue.severity` depends on the issue, not on the run it belongs to).
- No repeating groups: `selected_checks` is stored as an array type rather than as separate boolean columns (`seo_selected`, `ux_selected`, ...), avoiding a schema change every time a new check type is added.
- `issue` and `ux_suggestion` are deliberately *not* merged into one polymorphic table, even though doing so would reduce table count — merging them would require nullable columns that are only meaningful for one finding type (e.g. `lighthouse_audit_id` for issues, `observation/recommendation_id` for UX suggestions), which is itself a normalization smell and would blur the mechanical-vs-judgment distinction the rest of the system enforces.

8 Constraints

Constraint	Rationale
<code>fix.issue_id</code> NOT NULL	A fix must always trace back to a verifiable issue; the application layer never constructs a fix for a UX suggestion, and the schema enforces this rather than relying solely on application logic.
<code>audit_run.status</code> constrained to {running, completed, failed}	Prevents an invalid/typo'd status value from silently breaking status-based queries.
<code>report.run_id</code> UNIQUE	Enforces one report per run at the database level.
Foreign keys on <code>run_id</code> , <code>site_id</code> , <code>issue_id</code>	ON DELETE CASCADE is recommended for <code>issue</code> , <code>ux_suggestion</code> , <code>fix</code> , and <code>report</code> relative to <code>audit_run</code> , so that deleting a run cleans up its findings rather than leaving orphaned rows. <code>site</code> should <i>not</i> cascade-delete its runs by default, to preserve audit history intentionally.